

Generalized priming: why it is needed and why it is correct

Gal Kapsenel

September 30, 2026

Abstract

We show two things about Revizor’s priming step. First, the current implementation cannot report a whole class of genuine leaks: the ones whose only footprint falls outside the cache the tool reads. This is a property of the priming decision rule, not of any particular test case, and the canonicity leak we found is a concrete member of the class. Second, the proposed generalization does not change the violation definition, it only changes the methodology. It detects every leak the current tool detects and strictly more, and its localization step provably returns a genuine leaking input pair, distinguished by one and the same probe. We give two localizers, a linear scan and a binary search, and prove each optimal for what it returns.

1 Definitions

This section fixes every object the rest of the note uses; nothing here is proved. The proofs start in Section 2.

Definition 1 (Run and batch). A *test case* T is a fixed program. An *input* i sets the initial architectural state of the sandbox. A *run* starts from a microarchitectural state μ , executes T on i , and ends in a new microarchitectural state; we write $\text{run}(T, i, \mu)$. A *batch* runs a sequence of inputs in order. Before each run the architectural sandbox is set explicitly, but the microarchitectural state is *carried over*: the state entering a run is the one the previous run left behind. This carry-over is the only channel from one run to the next, and it flows only forward in time.

Definition 2 (Initialization sequence). An *initialization sequence* IS is a fixed routine whose job is to drive the microarchitectural state into one fixed, reproducible value μ_{IS} , regardless of what ran earlier.

Definition 3 (Traces). The *contract trace* $\text{ct}(T, i)$ is the leakage the reference contract permits for i under T ; it depends on T and i only. A run also leaves a microarchitectural footprint, and a *hardware trace* is the part of it an attacker reads. We name a generic hardware trace H , and two concrete ones matter here.

- H_{L1D} — the L1D cache trace measured by Prime+Probe or Flush+Reload, the hardware trace Revizor uses today.
- H_{full} — the *full* leftover a run leaves, of which H_{L1D} is one projection: anything H_{L1D} separates, H_{full} separates too, but not conversely.

Definition 4 (Violation). A *violation* is a tuple (T, i_0, i_1, μ) — a test case T , two distinct inputs i_0, i_1 , and a common microarchitectural state μ — such that the contract cannot tell the inputs apart yet the hardware does:

$$\text{ct}(T, i_0) = \text{ct}(T, i_1) \quad \text{and} \quad H(\text{run}(T, i_0, \mu)) \neq H(\text{run}(T, i_1, \mu)).$$

The faithful trace here is $H = H_{\text{full}}$; Revizor approximates it by its projection $H = H_{L1D}$.

Definition 5 (Priming, the current decision rule). Take a batch $i_0, i_1, \dots, i_k$ and write μ_p for the state reached after running $i_0, \dots, i_{p-1}$ (so input i_p runs from μ_p). Revizor *flags a suspected violation* between two same-class inputs i_a, i_b when they leave different L1D traces in their own slots:

$$\text{ct}(T, i_a) = \text{ct}(T, i_b) \quad \text{and} \quad H_{\text{L1D}}(\text{run}(T, i_a, \mu_a)) \neq H_{\text{L1D}}(\text{run}(T, i_b, \mu_b)).$$

Priming then equalizes history: it swaps the two inputs into each other's slot and re-measures each candidate against the other from a *common* preceding state. This yields two equalized comparisons,

$$\underbrace{H_{\text{L1D}}(\text{run}(T, i_a, \mu_a)) \text{ vs. } H_{\text{L1D}}(\text{run}(T, i_b, \mu_a))}_{\text{both from } \mu_a} \quad \underbrace{H_{\text{L1D}}(\text{run}(T, i_a, \mu_b)) \text{ vs. } H_{\text{L1D}}(\text{run}(T, i_b, \mu_b))}_{\text{both from } \mu_b}.$$

Priming *keeps* the finding if at least one comparison still differs, and *discards* it (declares a false positive) when both agree:

$$H_{\text{L1D}}(\text{run}(T, i_a, \mu_a)) = H_{\text{L1D}}(\text{run}(T, i_b, \mu_a)) \quad \text{and} \quad H_{\text{L1D}}(\text{run}(T, i_a, \mu_b)) = H_{\text{L1D}}(\text{run}(T, i_b, \mu_b)).$$

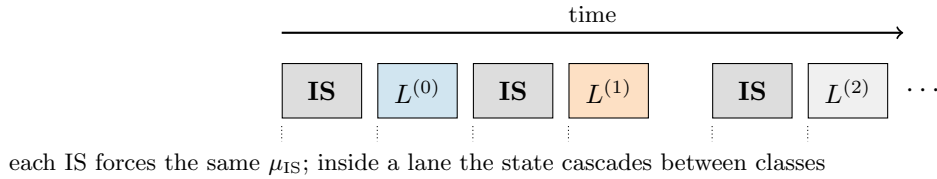
Definition 6 (Noncache violation). A violation (T, i_0, i_1, μ) (Definition 4) is *noncache* when

- (a) for every microarchitectural state μ' , $H_{\text{L1D}}(\text{run}(T, i_0, \mu')) = H_{\text{L1D}}(\text{run}(T, i_1, \mu'))$ — the choice between i_0 and i_1 never changes the L1D trace of its *own* run; and
- (b) for some μ'' , $H_{\text{full}}(\text{run}(T, i_0, \mu'')) \neq H_{\text{full}}(\text{run}(T, i_1, \mu''))$ — the two do leave different *full* footprints.

Definition 7 (Input class). Fix a test case T and contract trace ct . Two inputs i_0, i_1 are in the same *class* when they share a contract trace, $\text{ct}(T, i_0) = \text{ct}(T, i_1)$. A class is thus a set of inputs the contract deems indistinguishable.

Definition 8 (Boosting). Revizor *boosts* an input i (under T) by generating other inputs of the same class as i (Definition 7) — distinct inputs with the identical contract trace.

Definition 9 (Lane). A *lane* is a batch that holds one input from each of $k + 1$ classes: $L^{(\ell)} = (i_0^{(\ell)}, \dots, i_k^{(\ell)})$, where $i_m^{(\ell)}$ is its member of class m . Two lanes a, b therefore meet class m with two *different* inputs $i_m^{(a)}, i_m^{(b)}$ that still share the contract trace of class m ; boosting is what supplies the second member. Revizor runs lanes one after another, each preceded by IS (Definition 2):



The matrix picture. Stack the lanes as the rows of a matrix and let the classes be its columns. Entry (ℓ, m) is the input $i_m^{(\ell)}$. A lane is a row, read left to right; running it means IS, then the row's inputs in column order, with the microarchitectural state cascading along the row.

$$M = \begin{array}{c|cccc} & \text{class 0} & \text{class 1} & \cdots & \text{class } k \\ \hline \text{lane } a & i_0^{(a)} & i_1^{(a)} & \cdots & i_k^{(a)} \\ \text{lane } b & i_0^{(b)} & i_1^{(b)} & \cdots & i_k^{(b)} \\ \vdots & & & & \end{array}$$

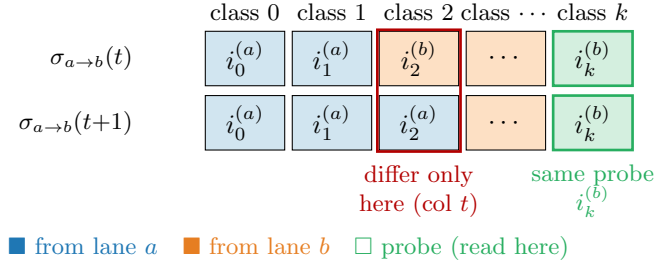
Every column is one class, so the contract cannot tell its entries apart. Any H_{L1D} difference between two rows read at the *same* column is therefore, by definition, a violation candidate.

Convention: the detecting class is the last one. Suppose a candidate is flagged at class d (two rows disagree at column d). A run at a later column happens *after* the class- d run, and carry-over flows only forward (Definition 1), so no later column can change the class- d reading. We may drop every column after d and relabel so the detecting class is the last one, k . From here on we call class k the *probe*.

Definition 10 (Prefix/suffix splice). For two lanes a, b and a split $t \in \{0, \dots, k+1\}$, the splice $\sigma_{a \rightarrow b}(t)$ is the row that takes columns *before* t from lane a and columns *from* t *on* from lane b :

$$\sigma_{a \rightarrow b}(t) = (\underbrace{i_0^{(a)}, \dots, i_{t-1}^{(a)}}_{\text{prefix: lane } a}, \underbrace{i_t^{(b)}, \dots, i_k^{(b)}}_{\text{suffix: lane } b}).$$

So $\sigma_{a \rightarrow b}(0) = L^{(b)}$ (all b) and $\sigma_{a \rightarrow b}(k+1) = L^{(a)}$ (all a). Two neighbours $\sigma_{a \rightarrow b}(t)$ and $\sigma_{a \rightarrow b}(t+1)$ are the same row except at column t : they share the prefix (columns $< t$, from a) and the whole suffix (columns $> t$, from b , probe included), and differ only in the input at column t . Each splice is run as a batch IS, $\sigma_{a \rightarrow b}(t)$, so both neighbours also share the same start μ_{IS} .

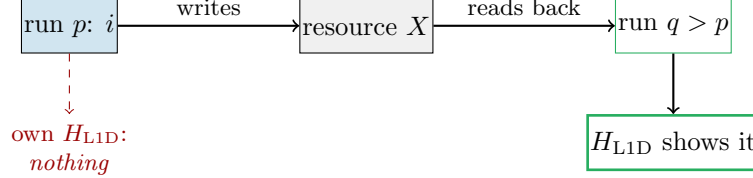


Definition 11 (Splice measurement). To *measure the splice* at split t : run the batch IS, $\sigma_{a \rightarrow b}(t)$ and read the L1D trace of its class- k (probe) run. Let μ_t be the microarchitectural state that IS and classes $0, \dots, k-1$ of $\sigma_{a \rightarrow b}(t)$ leave, and c_t the class- k input the splice carries ($c_t = i_k^{(a)}$ when $t = k+1$, otherwise $c_t = i_k^{(b)}$). The splice measurement is

$$r(t) = H_{\text{L1D}}(\text{run}(T, c_t, \mu_t)).$$

2 The limitation

Intuition. An input can *write* a microarchitectural resource X that its own run never *reads back* into the cache. Then that write leaves no mark on the run’s own H_{L1D} ; it survives only as leftover and shows up, if at all, in a *later* run that does read X . Current priming was built to erase exactly this kind of carried-over effect, so it erases the evidence of such a leak along with the noise.



Theorem 1 (Current priming misses noncache violations). *If a violation (T, i_a, i_b, μ) is non-cache (Definition 6), the rule of Definition 5 never reports it.*

Proof. A pair is reported only if it is first flagged as a suspected violation and then survives priming, so it suffices to show priming always discards it. Suppose the pair is flagged at slots a, b , with preceding states μ_a, μ_b . Priming runs the two equalized comparisons of Definition 5. By noncache condition 6(a), applied at $\mu' = \mu_a$ and at $\mu' = \mu_b$,

$$H_{\text{L1D}}(\text{run}(T, i_a, \mu_a)) = H_{\text{L1D}}(\text{run}(T, i_b, \mu_a)) \quad \text{and} \quad H_{\text{L1D}}(\text{run}(T, i_a, \mu_b)) = H_{\text{L1D}}(\text{run}(T, i_b, \mu_b)),$$

so both comparisons agree. That is exactly the discard condition, so priming declares a false positive and the pair is never reported. $\square$

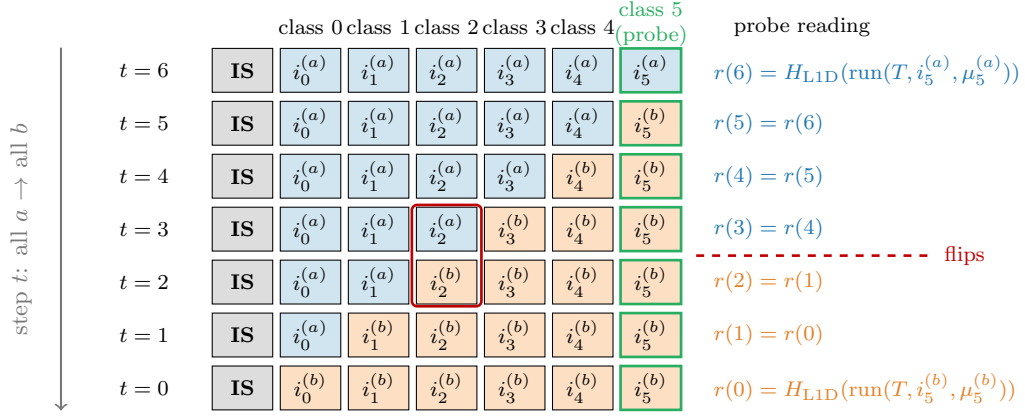
Remark 1. Theorem 1 is unconditional: it uses nothing about the channel or the test case, only the shape of the priming rule. Priming was introduced to discard differences caused by carried-over state, and a noncache violation is visible, if at all, only through carried-over state — so the rule discards it by construction.

The canonicity leak is such a violation. A BTB entry is indexed by (a subset of) a branch’s PC bits. Revizor’s test cases are loop-free, so within one run a given branch executes once: its BTB entry can be written but not re-read in the same run. Any information the input pushes into that entry, and that the cache cannot see by other means, therefore changes H_{full} but not the run’s own H_{L1D} . Conditions 6(a) and (b) hold, and by Theorem 1 current priming cannot report it. The branch history, the prefetcher, the TLB, the pattern history table and the return stack all supply the same kind of X .

3 Localizing the leak

We keep Definition 4 unchanged and only change the methodology. The key move is to *observe the leftover without reading it*: we never read H_{full} ; we let one input’s leftover propagate into the cache of a later run and read it there.

Assume Revizor found input class 5 from lanes a and b is a suspected violation, meaning, running lane a and measuring input class 5 returned a different H_{L1D} from running lane b and measuring at input class 5. That is $r(k+1) = H_{\text{L1D}}(\text{run}(T, i_5^{(a)}, \mu_5^{(a)})) \neq H_{\text{L1D}}(\text{run}(T, i_5^{(b)}, \mu_5^{(b)})) = r(0)$ where $\mu_5^{(a)}$ is the microarch state resulted by running lane a inputs prior to input class 5 and $\mu_5^{(b)}$ is the microarch state resulted by running lane b inputs prior to input class 5.



Notice that the ends of the above ladder are exactly the measurements Revizor flagged (Definition 11), so they disagree; hence the measurement must change at some step t^* . A step where it changes is a *boundary*. In the above example, the boundary is $t^* = 2$.

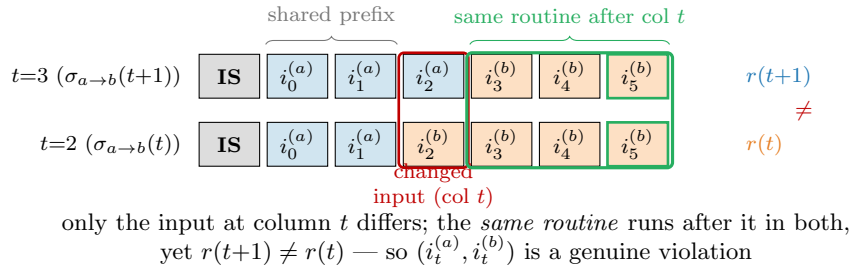
Recall $r(t)$ is the splice measurement (Definition 11). Two facts about boundaries settle everything: at least one exists (Proposition 1), and each one is a genuine violation (Lemma 1).

Proposition 1 (A boundary exists). *Whenever the ends disagree ($r(k+1) \neq r(0)$), some split t is a boundary.*

Proof. Stepping gives the chain $r(0), r(1), \dots, r(k+1)$ with $r(0) \neq r(k+1)$ (the two ends are the flagged candidate). If $r(t) = r(t+1)$ for all t then $r(0) = \dots = r(k+1)$, contradicting the ends. Hence $r(t) \neq r(t+1)$ for some t — a boundary. $\square$

Lemma 1 (Every boundary is a genuine violation; noncache when H_{L1D} -blind). *Let t be any boundary, i.e. $r(t) \neq r(t+1)$. Then $(T, i_t^{(a)}, i_t^{(b)})$ is a genuine violation (Definition 4). If moreover the pair is indistinguishable in H_{L1D} for all μ , the violation is noncache.*

The boundary is a single step, so the two batches around it differ in exactly one input; after that input, the *exact same routine* runs in both — the identical suffix, whose last run is the probe that is measured. That is what makes the returned pair a genuine violation. We zoom into the two boundary rows $t = 3$ and $t = 2$ of the ladder above:



Proof. The neighbours $\text{IS}, \sigma_{a \rightarrow b}(t)$ and $\text{IS}, \sigma_{a \rightarrow b}(t+1)$ share the same IS (start μ_{IS}), the prefix (columns $< t$, lane a) and the whole suffix (columns $> t$, lane b , probe included) — the same routine runs after column t in both; they differ only at column t . Their probe readings differ (that is the boundary). So one and the same routine, run after an identical history except for the single input at column t , already separates $i_t^{(a)}$ from $i_t^{(b)}$.

Hence $i_t^{(a)}, i_t^{(b)}$ are distinct inputs of one class (they share a contract trace); they run from a common preceding state (the shared IS and prefix), the equalization Definition 5 asks for; and they leave different microarchitectural state behind, since it feeds the differing probe (H_{full} differs). That is a violation.

If in addition $i_t^{(a)}, i_t^{(b)}$ have the same H_{L1D} for all μ , then 6(a) holds, and 6(b) holds because localization has just separated their downstream effect. The violation is noncache: unseen by current priming, yet certified here through its effect on the probe, without ever reading H_{full} . $\square$

Together the two facts settle correctness: a boundary always exists, and *any* boundary the sweep exposes is a genuine violation. So localization reduces to finding a boundary — the two localizers below differ only in *which* boundary they return.

Remark 2 (Several boundaries may exist). The chain $r(0), \dots, r(k+1)$ can change value more than once, so there may be several boundaries — up to $k+1$ of them — each a distinct leaking class and each genuine by Lemma 1. The linear scan can report them all; the binary search returns just one.

Remark 3 (Self- vs. cross-input, from the boundary t). The boundary column t says *what kind* of leak it is.

- $t = k$ (the probe column itself): the pair leaks about *itself* — swapping the probe input changes the probe reading in the ordinary, same-run way. This is a *self-dependent* leak, the kind current priming already catches.
- $t < k$: an *earlier* class’s input changes a *later* reading — a *cross-input* leak, the noncache case that motivates this work.

A caller that wants only cross-input findings drops $t = k$ and keeps $t < k$; every kept boundary is still genuine by Lemma 1.

Both directions. Within one direction the probe is fixed, but which class actually reads the leftover back into the cache can be lane-specific: perhaps only $i_k^{(a)}$ performs the readout, or only some class in a lane- a suffix. So we step both ways: $a \rightarrow b$ probes with $i_k^{(b)}$ and a lane- b suffix, $b \rightarrow a$ with $i_k^{(a)}$ and a lane- a suffix; running both covers either. Each direction is independently sound (Lemma 1), so reporting a boundary found in either adds no false positive.

3.1 Linear scan

Definition 12 (Localization by linear scan). In direction $a \rightarrow b$:

1. Read $r(k+1)$ (all a). Then step down, $t = k, k-1, \dots, 0$, reading $r(t)$.
2. After each step compare the neighbours $r(t)$ and $r(t+1)$: they come from the same batch IS, σ apart from the single input at column t .
3. Report the first (largest) t with $r(t) \neq r(t+1)$ — or every such t if all leaking classes are wanted — with the pair $i_t^{(a)}, i_t^{(b)}$.

Run the same in direction $b \rightarrow a$. Every reported pair is a genuine violation by Lemma 1.

Remark 4 (Measurement). A single probe reading is noisy, so each $r(t)$ is a *repeated* measurement, and two readings count as different only when they stay apart across the repetitions. Enough repetitions push the noise down until $r(t)$ behaves as a stable value, which is what the steps above compare. The other fact we use is architectural: the sandbox is set before each run and each lane starts from μ_{IS} , so the only channel between columns of a spliced batch is the forward microarchitectural cascade.

Cost and a matching lower bound

We measure a localizer by its *runs*: one run executes one spliced batch and returns its probe reading $r(t)$; the only promise is $r(0) \neq r(k+1)$. We drop the constant repetition count (Remark 4) and the factor two for the two directions. Write $t_{\max} = \max\{t : r(t) \neq r(t+1)\}$, the largest leaking class.

Proposition 2 (Cost of the scan). *The top-down scan reads $r(k+1), r(k), \dots$ until the first change and returns $t_{\max}$; in the worst case it uses $k+2$ runs.*

Proof. Reading downward, the first t with $r(t) \neq r(t+1)$ is the largest such t , and along the way it has confirmed $r(t+1) = \dots = r(k+1)$, so no boundary lies above it. The values read are $r(k+1), \dots, r(t)$, at most all $k+2$ of them (attained when $t_{\max} = 0$). $\square$

Theorem 2 (Finding $t_{\max}$ needs a linear scan). *Any algorithm that always returns $t_{\max}$ makes at least k runs in the worst case. The scan is therefore optimal up to a constant.*

Proof. Fix $t^* \in \{0, \dots, k\}$ and the single-boundary chain S_{t^*} with $r(x) = r_b$ for $x \leq t^*$ and $r(x) = r_a$ for $x > t^*$ (two distinct values). Its only boundary is t^* , so $t_{\max} = t^*$, and correctness forces the output t^* on S_{t^*} .

Suppose the algorithm leaves some $u \in (t^*, k]$ unread. Build S' from S_{t^*} by setting $r(u) = c$ for a third value $c \notin \{r_a, r_b\}$. Since S' agrees with S_{t^*} on every position read, the execution is identical and it again outputs t^* . But $u \leq k$ gives $r(u+1) = r_a$, so $r(u) = c \neq r(u+1)$ is a boundary above t^* , whence $t_{\max}(S') \geq u > t^*$: wrong. Hence every $u \in (t^*, k]$ is read, that is $k - t^*$ runs; taking $t^* = 0$ gives at least k . $\square$

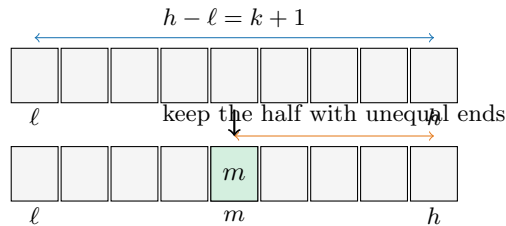
4 A logarithmic localizer: binary search

The scan returns the largest leaking class. If any one leaking class suffices, a binary search finds one far faster, using the same splices and the same probe. It keeps a window $[\ell, h]$ whose ends disagree ($r(\ell) \neq r(h)$) and always narrows to a half that still disagrees, so a boundary is trapped inside.

Definition 13 (Localization by binary search). In direction $a \rightarrow b$, keep an interval $[\ell, h]$ with $r(\ell) \neq r(h)$; start $\ell = 0, h = k+1$, which holds by the candidate. While $h > \ell + 1$, let $m = \lfloor (\ell + h)/2 \rfloor$, run $\text{IS}, \sigma_{a \rightarrow b}(m)$, read $r(m)$, and

- if $r(m) \neq r(\ell)$, set $h := m$;
- otherwise $r(m) = r(\ell) \neq r(h)$, so set $\ell := m$.

When $h = \ell + 1$, return $t = \ell$ with the pair $i_\ell^{(a)}, i_\ell^{(b)}$.



Proposition 3 (Binary search returns a genuine violation). *Binary search terminates after at most $\lceil \log_2(k+1) \rceil$ runs and returns a boundary t . By Lemma 1 the pair $i_t^{(a)}, i_t^{(b)}$ is a genuine violation.*

Proof. The invariant $r(\ell) \neq r(h)$ holds initially and is preserved: if $r(m) \neq r(\ell)$ the new interval $[\ell, m]$ has unequal ends; otherwise $r(m) = r(\ell)$, and since $r(\ell) \neq r(h)$ we get $r(m) \neq r(h)$, so $[m, h]$ has unequal ends. Each step halves $h - \ell$, from $k + 1$ to 1, so it stops after at most $\lceil \log_2(k + 1) \rceil$ reads with $h = \ell + 1$ and $r(\ell) \neq r(\ell + 1)$, a boundary. Lemma 1 applies. $\square$

Binary search returns *some* leaking class, not necessarily $t_{\max}$, and it does not enumerate the others; when that is all one needs, it is much cheaper.

Theorem 3 (Finding any boundary needs a logarithmic search). *Any algorithm that always returns some boundary makes at least $\lceil \log_2(k + 1) \rceil$ runs in the worst case. Binary search is therefore optimal.*

Proof. Use the single-boundary chains S_j , $j \in \{0, \dots, k\}$, with $r(x) = r_b$ for $x \leq j$ and $r(x) = r_a$ for $x > j$; each satisfies $r(0) \neq r(k + 1)$ and has its unique boundary at j . Let C be the set of j still consistent with the answers so far; initially $|C| = k + 1$. A run at position t returns r_b for chains with $j \geq t$ and r_a for those with $j < t$, splitting C into $\{j \in C : j \geq t\}$ and $\{j \in C : j < t\}$; the adversary answers with the value of the larger part, so each run leaves $|C|$ at least half its previous size. The algorithm may stop only when $|C| = 1$: if two chains S_{j_1}, S_{j_2} remain, its single output is the boundary of at most one, so it is wrong on the other. Reducing $|C|$ from $k + 1$ to 1 by halving needs at least $\lceil \log_2(k + 1) \rceil$ runs. $\square$

5 Comparison

Both localizers use the same splices, the same probe comparison, and the same soundness (Lemma 1); they differ in what they return and in cost. Entries are per direction; multiply by two for both directions and by the repetition count of Remark 4.

	Linear scan	Binary search
Runs (worst case)	$k + 2$	$\lceil \log_2(k + 1) \rceil$
Matching lower bound	$\geq k$ runs (Thm. 2)	$\geq \lceil \log_2(k + 1) \rceil$ runs (Thm. 3)
Returns	the largest leaking class $t_{\max}$; all leaking classes if continued	one leaking class (some boundary)
Soundness	every returned pair is a genuine violation	every returned pair is a genuine violation
Completeness	finds every leaking class	finds one, may miss others

In short, current priming cannot report a noncache violation (Theorem 1), while the lane-toggle localization certifies one as a genuine violation, adhering to the existing definitions, distinguished by one identical probe (Lemma 1). The generalization keeps Definition 4 and detects a strict superset. When every leaking class is wanted, the linear scan is optimal (Theorem 2); when one suffices, binary search is optimal (Theorem 3).